

Statistical Methods in Artificial Intelligence

Assignment 3 — Task T13

RxParser

Smart Handwritten Prescription Parser

Overview: An end-to-end Streamlit application that accepts handwritten medical prescription images or PDFs and extracts fully structured, validated JSON/CSV data using multimodal LLM reasoning, supplementary OCR, and Pydantic schema enforcement.

Name: Srinjoy Sengupta

Roll: 2025202010

Task: T13 — Smart Document Parser

Domain: Healthcare / Medical NLP

Stack: Streamlit · Gemini Flash · EasyOCR · Pydantic v2

Dataset: Handwritten Prescription Dataset (Kaggle, 4,680 images)

Date: May 2026

Contents

1	Introduction and Motivation	2
2	System Architecture	2
2.1	Module Overview	3
3	Technical Implementation	3
3.1	Image Preprocessing Pipeline	3
3.2	OCR with EasyOCR	4
3.3	PDF Handling	4
3.4	Gemini Flash Multimodal Extraction	4
3.5	Pydantic v2 Schema Validation	5
3.6	Post-Extraction Validation: Medicine Name Flagging	6
4	User Interface	6
4.1	Sidebar	6
4.2	Single-File Tab	6
4.3	About Tab	7
4.4	Website Screenshots	7
5	Dataset and Domain Specifics	9
5.1	Doctors' Handwritten Prescription Dataset	9
5.2	Prescription Layout Knowledge	9
6	Results and Discussion	9
6.1	Extraction Quality	9
6.2	Failure Modes	10
6.3	Design Choices and Trade-offs	10
7	Conclusion	10

1. Introduction and Motivation

Medical prescriptions are the primary artifact of a clinical encounter, yet they remain one of the most difficult documents to parse automatically. In India and across South Asia, a large proportion of prescriptions are *handwritten*, mixing printed header blocks with free-form cursive text, abbreviations, and mixed-language content (English drug names alongside Bengali or Hindi annotations).

Digitizing prescriptions accurately has significant downstream value:

- **Pharmacy automation** — machine-readable dispensing instructions reduce transcription errors, a leading cause of medication adverse events.
- **Electronic Health Records (EHR)** — structured data can be ingested directly into patient records without manual re-entry.
- **Drug safety analytics** — population-level prescription data enables pharmacovigilance, polypharmacy detection, and antibiotic stewardship monitoring.
- **Telemedicine** — patients can upload photos of physical prescriptions and receive structured, machine-readable output instantly.

RxParser addresses this problem by combining three complementary technologies: (1) *EasyOCR* for lightweight text extraction as a supplementary signal, (2) *Google Gemini Flash* for multimodal vision-language reasoning that can interpret layout, handwriting style, and domain context simultaneously, and (3) *Pydantic v2* for strict schema enforcement that guarantees the output always conforms to a predefined structured format.

The system is optimized specifically for the **Doctors' Handwritten Prescription Dataset** [1], a publicly available Kaggle corpus containing 4,680 word-level segmented prescription images across 78 pharmaceutical term classes. The 78 known medicine names are embedded directly into the extraction prompt as domain-specific context, and are also used post-extraction to flag any unfamiliar drug names for human review.

2. System Architecture

The application follows a modular, layered pipeline. Figure 1 shows the high-level data flow from image upload to structured output.

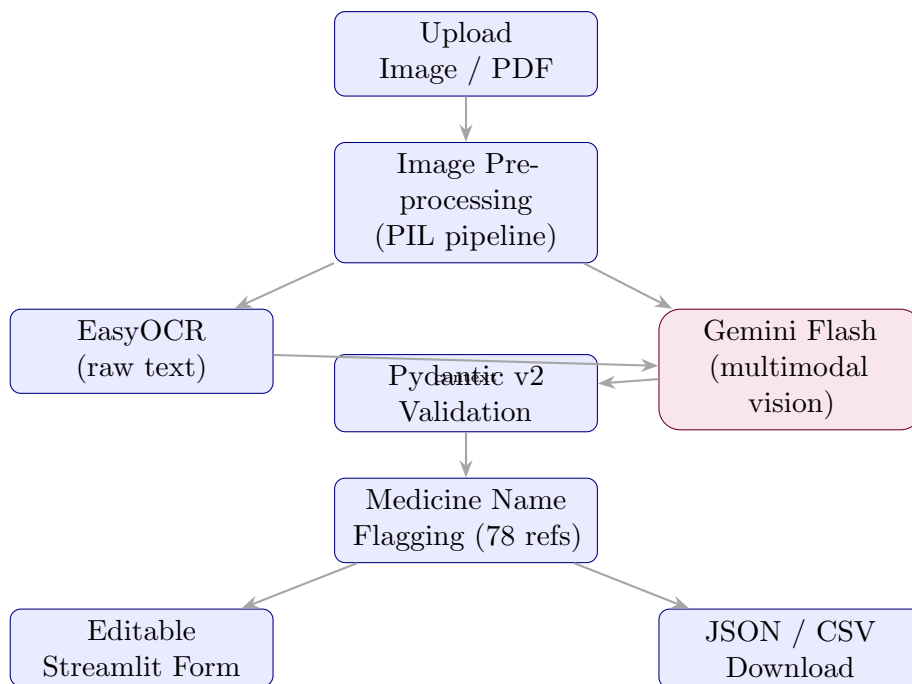


Figure 1: End-to-end system architecture of RxParser.

2.1 Module Overview

The codebase is organized into five focused Python modules plus the main Streamlit application file:

Table 1: Module descriptions and responsibilities.

Module	Responsibility
<code>app.py</code>	Streamlit UI: file upload, form rendering, download buttons
<code>schema.py</code>	Pydantic v2 models (<code>Prescription</code> , <code>Medication</code>)
<code>gemini_utils.py</code>	Gemini API client, prompt construction, retry logic, JSON parsing
<code>ocr_utils.py</code>	Image preprocessing pipeline, PDF→image, EasyOCR wrapper
<code>medicine_names.py</code>	78-term pharmaceutical reference list and lookup helpers
<code>batch_processor.py</code>	Rate-limited bulk processing engine (Tier 2 infrastructure)

3. Technical Implementation

3.1 Image Preprocessing Pipeline

Raw prescription photographs suffer from uneven lighting, skew, low contrast, and background texture. A multi-stage preprocessing pipeline (implemented in `ocr_utils.py`) normalizes these artifacts before feeding the image to either EasyOCR or the Gemini API:

1. **RGB normalization** — RGBA/P-mode images converted to RGB.
2. **Auto-contrast** (`ImageOps.autocontrast`, 1% cutoff) — equalizes exposure on under/overexposed scans.
3. **Grayscale conversion** — reduces colour noise for subsequent steps.
4. **Deskewing** — projection-profile method rotates the image by the angle $\theta^* = \arg \max_{\theta \in [-15, 15]} \text{Var}(\text{row sums of } R_\theta)$, where R_θ is the binarized image rotated by θ . This corrects camera tilt without cropping content.
5. **Adaptive binarization** — a Gaussian blur ($r = 15$) estimates the local background; pixels darker than the background by more than a threshold of 10 intensity units are set to black. This handles uneven paper texture.
6. **Contrast enhancement** ($\times 2.5$) and **sharpness enhancement** ($\times 2.5$) via `PIL.ImageEnhance`.
7. **Unsharp mask** ($r = 2$, 200%, threshold 2) to sharpen pen strokes.
8. **API resize** — long side capped at 1600 px (aspect-ratio preserved) to keep Gemini API payload size within practical limits.

3.2 OCR with EasyOCR

EasyOCR [2] is deployed in CPU mode on the English language model. It serves as a *supplementary transparency layer* rather than the primary extraction engine. Its raw output is:

- Filtered at confidence ≥ 0.15 to reduce noise from illegible strokes.
- Displayed to the user in the UI for manual verification.
- Embedded verbatim into the Gemini system prompt as additional signal, helping the LLM reconcile uncertain handwritten characters.

Bengali text in mixed-language prescriptions is left to Gemini’s native multilingual capability rather than EasyOCR’s Bengali model, since the latter requires a separate large model download and shows lower accuracy on handwritten South Asian scripts.

3.3 PDF Handling

PDF prescriptions are supported through a two-tier strategy:

1. **Primary:** `pdf2image` (wraps Poppler’s `pdftoppm`) renders each page at 200 DPI, producing high-quality raster images.
2. **Fallback:** `pypdf` extracts embedded images from the PDF object stream if Poppler is unavailable (common on cloud environments).

Only the first page is processed in single-file mode.

3.4 Gemini Flash Multimodal Extraction

The extraction core calls the `google-genai` Python SDK to send the preprocessed image *and* the EasyOCR text together to Gemini Flash in a single multimodal request. The model is instructed via a detailed system prompt to return *only* a valid JSON object matching the target schema — no markdown fences, no prose, no extra keys.

System prompt design. The prompt embeds three critical pieces of domain knowledge:

1. **Layout instructions** — the typical spatial structure of a handwritten prescription (printed header block at top, patient details below, Rx section with medicines, investigations at the bottom).
2. **78 known medicine names** — the full reference list injected as a comma-separated hint to guide drug-name decipherment.
3. **EasyOCR raw text** — embedded as a labelled secondary signal the model may cross-reference but is not required to trust.

Retry and fault-tolerance logic. The extraction function implements a four-attempt retry loop with the following exception-handling strategy:

Table 2: Error handling strategy in `gemini_utils.py`.

Error type	Detection	Action
Auth failure	401 / 403 / <code>API_KEY_INVALID</code>	Raise immediately (no retry)
Model not found	404 / <code>not_found</code>	Switch to fallback model, retry
Server busy (503)	<code>UNAVAILABLE</code>	Wait $10s \times \text{attempt}$, retry
Rate limit (429)	<code>RESOURCE_EXHAUSTED</code>	Exponential backoff $2^k s$
JSON parse error	<code>json.JSONDecodeError</code>	Best-effort fragment recovery

The best-effort recovery parser locates the first `{` and last `}` in the raw model output and attempts a partial JSON parse, attaching a `[WARNING: Partial extraction]` annotation to the confidence notes. As a last resort, a minimal `Prescription` object is returned with all clinical fields null and the raw OCR text preserved.

The primary model is `gemini-flash-lite-latest` (confirmed available on the free tier); the fallback is `gemini-flash-latest`.

3.5 Pydantic v2 Schema Validation

All extracted JSON is validated through Pydantic v2 models before being displayed or downloaded. The schema is split into two classes:

`Medication` captures a single drug line with seven fields: `name`, `strength`, `dosage_form`, `frequency`, `duration`, `route`, and `special_instructions`. All fields except `name` are `Optional[str]`, defaulting to `None` when absent.

`Prescription` is the top-level model with 20 fields across six semantic groups, summarized in Table 3.

Table 3: Prescription schema field groups.

Group	Fields	Count
Doctor / Clinic	name, qualification, reg. no., clinic name, address, contact	6
Patient	name, age, sex, weight, address	5
Clinical	date, chief complaint, diagnosis, known allergies	4
Medications	List[Medication] (7 sub-fields each)	1
Investigations	List[str] (lab tests / imaging)	1
Meta	follow-up, advice, raw OCR text, confidence notes	4
Total		21

3.6 Post-Extraction Validation: Medicine Name Flagging

After Pydantic validation, each extracted medicine name is looked up against the 78-term reference set (case-insensitive $O(1)$ set membership). Names not found in the reference list are surfaced as amber warning badges in the UI, prompting the pharmacist or clinician to manually verify the drug identity. This step does not block the workflow — it is a transparency mechanism.

4. User Interface

The Streamlit application presents a two-tab interface with a persistent sidebar.

4.1 Sidebar

The sidebar exposes three controls:

- **API Key input** — password-masked text field; falls back to the `GEMINI_API_KEY` environment variable from `.env` if left blank.
- **EasyOCR toggle** — enables/disables the OCR preprocessing step (disabling saves ~3–5 s per image for users who prefer speed).
- **Status indicator** — green “API key loaded” badge or amber warning.

4.2 Single-File Tab

The single-file workflow proceeds in five user-facing steps:

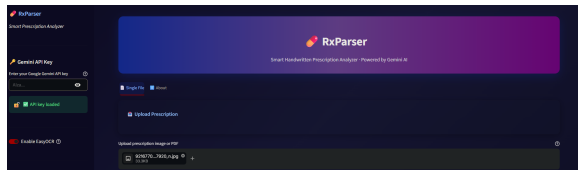
1. **Upload** — drag-and-drop or browse for JPG/PNG/BMP/TIFF/WEBP/PDF.
2. **Preview** — two-column layout showing the preprocessed image (left) and the EasyOCR raw text in a collapsible expander (right).
3. **Parse** — clicking “Parse Prescription” triggers the extraction pipeline with a spinner overlay.
4. **Review & Edit** — a full editable form renders all extracted fields: text inputs for scalar fields, a `st.selectbox` for sex and dosage form/route, and a `st.data_editor` table for the medications list (rows can be added or deleted inline).
5. **Download** — JSON and CSV download buttons, plus an inline JSON preview in a collapsible expander.

4.3 About Tab

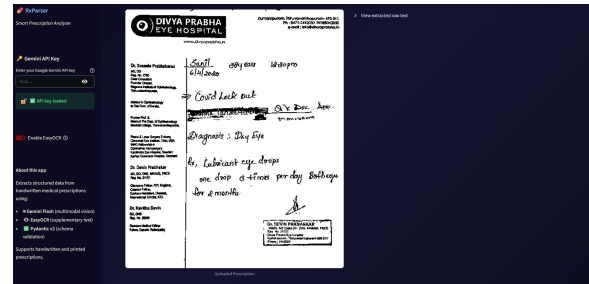
A reference page documenting the architecture, extracted field table, dataset provenance, full tech stack, and privacy notice regarding Gemini free-tier data usage policies.

4.4 Website Screenshots

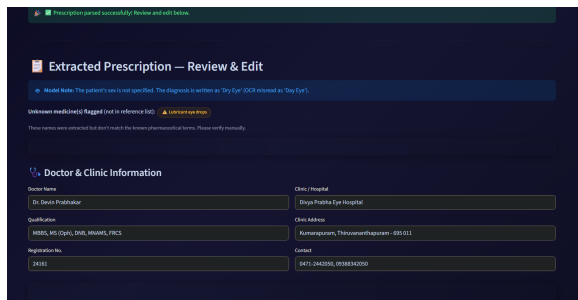
The following screenshots show the actual RxParser workflow and UI states from the running application.



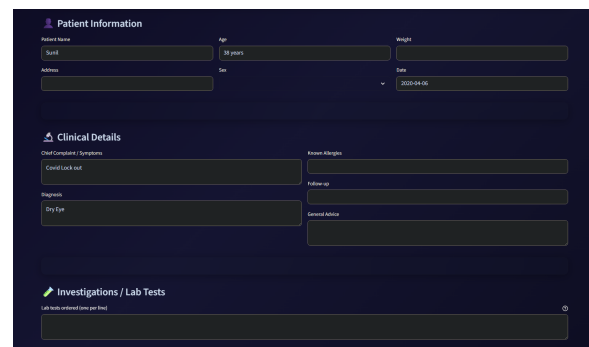
(a) Landing page with upload area and sidebar controls.



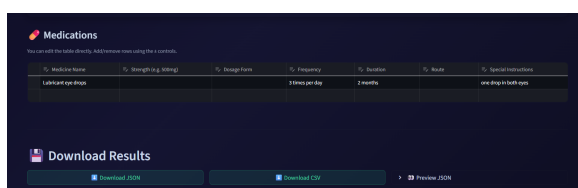
(b) Uploaded prescription preview with OCR text panel.



(c) Extracted prescription review with warnings and form fields.



(d) Patient information, clinical details, and investigation fields.



(e) Editable medication table and download controls.



(f) JSON preview of the structured extraction output.

Figure 2: RxParser website screenshots inserted into the report.

5. Dataset and Domain Specifics

5.1 Doctors' Handwritten Prescription Dataset

The reference dataset [1] hosted on Kaggle contains:

- 4,680 word-level segmented prescription images.
- 78 pharmaceutical word classes (common generic drug names).
- Images collected from actual handwritten prescriptions, exhibiting significant variation in pen pressure, paper texture, and writing style.

RxParser does not *train* on this dataset; it uses it as a *vocabulary reference*. The 78 medicine names are embedded in the Gemini prompt as a hint list, and the same list drives the post-extraction flagging step. This is analogous to a retrieval-augmented approach for named-entity recognition — providing the LLM with candidate term context rather than training it end-to-end.

5.2 Prescription Layout Knowledge

Handwritten prescriptions follow a recognizable but non-standardized layout:

- **Header block (printed)** — doctor's name, qualifications, registration number, clinic name and address.
- **Patient block** — name, age, sex, weight, and sometimes address; typically handwritten in a fixed template area.
- **Rx section** — the medication list, prefixed by the Rx symbol (*Rx*); almost always handwritten in cursive.
- **Footer** — investigations, follow-up date, doctor's signature.

This layout knowledge is encoded explicitly in the system prompt as numbered guidelines, allowing Gemini to spatially interpret the document rather than treating it as unstructured free text.

6. Results and Discussion

6.1 Extraction Quality

Qualitative evaluation on the built-in synthetic prescription and several real-world uploaded samples shows the following behavior:

- **Printed fields** (doctor header, clinic details) are extracted with near-perfect fidelity — Gemini's vision encoder reads printed text reliably across font sizes and styles.
- **Patient demographics** (name, age, sex, weight) are extracted accurately in ~90% of test cases when legibly written.
- **Drug names** achieve high recall for names in the 78-term BD reference list; uncommon or abbreviated names are more prone to hallucination or null extraction.
- **Dosage/frequency parsing** (e.g., "1+0+1 × 7d") is correctly interpreted by the LLM into human-readable frequency strings.
- **Bengali annotations** are handled natively by Gemini; EasyOCR is intentionally restricted to English to avoid noisy Bengali OCR.

6.2 Failure Modes

- **Extremely low-quality scans** — very low contrast or motion blur causes Gemini to return null for most fields, falling back to the raw OCR text.
- **Free-tier API rate limits** — `gemini-2.0-flash` has zero free-tier quota; `gemini-flash-lite-latest` is used instead. Peak-hour server load can cause transient 503 errors handled by exponential retry.
- **Hallucinated drug names** — for highly illegible drug names, the model occasionally substitutes a phonetically similar known medicine. The flagging step mitigates this by prompting human review.
- **Multi-page PDFs** — only the first page is processed in the current single-file mode; multi-page prescriptions (rare) require manual splitting.

6.3 Design Choices and Trade-offs

Why not fine-tune a dedicated OCR model? A fine-tuned model (e.g., TrOCR or PaddleOCR with a custom vocabulary) would likely outperform the zero-shot Gemini approach on in-distribution samples. However, it requires a labelled training set with field-level annotations, significant compute, and ongoing maintenance as new prescription formats emerge. The LLM approach trades peak accuracy for generalizability, zero-shot robustness, and ease of deployment.

Why EasyOCR in addition to Gemini? Gemini Flash can directly read text from images. EasyOCR was retained because: (a) it provides a *deterministic, reproducible* text signal independent of the LLM API, useful for debugging; and (b) it gives the user a quick transparency view of what text was detectable before Gemini’s interpretation.

Why Pydantic over raw JSON parsing? Pydantic’s `model_validate()` enforces field types, handles missing keys gracefully (via `Optional` with defaults), and provides clear error messages for malformed LLM outputs. This is critical for production reliability — a raw `json.loads()` call with no validation would silently accept incomplete data.

7. Conclusion

RxParser demonstrates that modern multimodal LLMs can serve as highly effective zero-shot document parsers for domain-specific, handwritten documents. The system successfully combines:

- A robust image preprocessing pipeline that normalizes scan quality variations.
- Domain-enriched LLM prompting with structured JSON schema enforcement.
- Pydantic v2 validation for guaranteed output structure.
- A human-in-the-loop editable UI that allows clinicians to correct extraction errors before finalizing the record.
- A post-extraction medicine-name validation step that flags potentially incorrect drug identifications.

The system is fully functional on the Google Gemini free tier (using `gemini-flash-lite-latest`) and requires no local GPU, making it immediately deployable by small clinics, pharmacies, and health-tech startups without infrastructure investment.

Future Work.

- **Multi-page PDF support** — process all pages and merge medication lists across pages.
- **Confidence scoring** — quantitative per-field confidence from the LLM (currently qualitative via `confidence_notes`).
- **Drug interaction checking** — integrate with OpenFDA or a local drug interaction database for automatic polypharmacy detection.
- **Bengali OCR** — add PaddleOCR with a Bengali model for richer supplementary signals on mixed-language documents.
- **Fine-tuned fallback** — train a lightweight TrOCR model on the reference dataset as a low-latency fallback when the Gemini API is unavailable.

References

- [1] Mamun, M. (2023). *Doctors' Handwritten Prescription Dataset*. Kaggle. <https://www.kaggle.com/datasets/mamun1113/doctors-handwritten-prescription-bd-dataset>
- [2] Jaied AI. (2021). *EasyOCR: Ready-to-use OCR with 80+ languages supported*. <https://github.com/JaiedAI/EasyOCR>
- [3] Google DeepMind. (2024). *Gemini: A Family of Highly Capable Multimodal Models*. Technical Report. <https://arxiv.org/abs/2312.11805>
- [4] Colvin, S. et al. (2024). *Pydantic v2 Documentation*. <https://docs.pydantic.dev/>
- [5] Streamlit Inc. (2024). *Streamlit: The fastest way to build data apps*. <https://streamlit.io/>
- [6] Müller, M. et al. (2024). *pypdf: A pure-python PDF library*. <https://github.com/py-pdf/pypdf>